ISE-A2

Domain Model and Testable Specification

Two-Stage Clinic Appointment and Arrival-Confirmation Workflow

Project continuity from ISE-A1

This specification carries forward the same project idea: patients book a configurable arrival window, confirm physical arrival at a kiosk or assisted desk, and enter a visible queue. The scope is non-emergency outpatient coordination in China; the model is proposed, not approved by a specific clinic.

Course Intelligent Software Engineering

Assessment ISE-A2 | Domain Model and Testable Specification

Maximum marks 5

Release 5 October 2026, 16:30 (Asia/Shanghai)

Deadline 12 October 2026, 16:30 (Asia/Shanghai)

Prepared 12 October 2026

Data boundary Synthetic examples only; no patient data or clinical records

Engineering principle

Booking is not proof of arrival. Arrival confirmation is not a clinical decision. Every state change must be authorised, traceable and independently checkable.

Result Summary (150–300 words)

This report carries the ISE-A1 clinic appointment project into a formal domain model and testable specification. The proposed system coordinates non-emergency outpatient appointments using a clinic-configured arrival window followed by on-site check-in and queue-ticket issuance. The domain vocabulary distinguishes Appointment, ClinicWindow, QueueTicket and AuditEvent, and separates the appointment lifecycle from the queue-ticket lifecycle. This avoids treating a booking confirmation as proof that a patient is physically present.

The process and state models define booking, cancellation, check-in, late-arrival review, no-show reconciliation, calling and consultation completion. The data dictionary records types, meanings, trusted authorities, validation rules and privacy boundaries. Six invariants cover valid time windows, appointment/time consistency, one active ticket per appointment, ticket issuance only after accepted check-in, authorised no-show recording, and legal queue-state transitions. A decision table makes the arrival-window boundaries explicit: the start is inclusive, the end is exclusive, and reaching the end of the grace period makes a booking eligible for staff review rather than automatically declaring a no-show. The 15-minute grace period is a configurable prototype assumption, not a known clinic policy.

A small Python reference model was checked with 36 unit tests using synthetic examples; all 36 passed in the recorded run. This demonstrates internal consistency of the selected rules, not clinical effectiveness or production readiness. Public Chinese outpatient research and official privacy/cybersecurity materials inform the context, but no target-clinic data, interview, pilot or stakeholder validation is claimed. A real deployment would require clinic approval of policies, access roles, retention and fallback procedures.

Word count: 248

Document map

Section	Contents
1	A1 continuity, problem boundary and assumptions
2	Domain glossary, entities, value objects and context map
3	Domain events and process flow
4	Appointment and queue state models
5	Data dictionary and data authority
6	Invariants, violations and automated checks
7	Decision table and formal contract
8	Data source, quality, retention, deletion and privacy
9	Scenario walkthrough record and counterexamples
10	AI use, source verification, limitations and references
Appendix A	Execution environment and test result
Appendix B	Complete reference model and unit-test source listing

1. Connection to ISE-A1 and Scope

ISE-A1 established the project opportunity: clinics may need a clearer connection between online booking, actual arrival and queue coordination. ISE-A2 does not repeat the problem investigation. It formalises the workflow into shared terminology, states, data fields and rules that can be checked independently.

1.1 Problem boundary

The bounded context is appointment coordination and queue visibility for planned, non-emergency outpatient visits. The system accepts a booking into an available clinic-defined time window, records a valid arrival confirmation, issues at most one active queue ticket for the appointment, and exposes the current queue state to authorised clinic staff.

Out of scope by design

Clinical triage, diagnosis, treatment, electronic medical records, clinical priority ranking, billing/insurance, prescription management, automated overbooking and a production wait-list algorithm are excluded. The system must not delay emergency care: urgent cases follow the clinic's existing clinical/emergency process, not this appointment queue.

1.2 Working assumptions

- The initial setting is a hypothetical outpatient clinic in China; a specific facility and its real workflow have not been verified.
- A clinic manager or authorised scheduling role configures services, capacity and arrival windows. Example windows such as 09:00–11:00 are illustrative.
- All timestamps are timezone-aware. The clinic's display timezone is Asia/Shanghai; the backend must make timezone handling explicit.
- The candidate interval is half-open: [window_start, window_end). The start is included; the end is outside the window.
- A 15-minute grace period is used as a configurable test value only. It is not presented as a national rule or clinic-approved policy.
- Kiosk failure and patients who need assistance must have a clinic-approved assisted/downtime path. The model does not assume everyone has a smartphone.
- Prototype tests use synthetic records only; no real patient information is collected or needed.

1.3 Design goals

Goal	Specification consequence
Separate reservation from arrival	An appointment stays CONFIRMED until the server accepts check-in; a booking alone cannot create a queue ticket.
Make decisions explicit	Early, in-window, late-review and no-show-eligible outcomes are different results.
Prevent duplicate queue entries	A repeated kiosk request returns the existing ticket or is safely rejected; it must not create a second active ticket.
Keep clinical decisions out of scope	The queue service records operational status and does not decide urgency or treatment priority.
Minimise and protect data	Use a pseudonymous patient reference and a booking reference; do not copy diagnoses or clinical notes into queue records.

2. Domain Glossary, Entities and Context

2.1 Shared vocabulary

Term	Kind	Definition and rule
Appointment	Entity / aggregate	A reservation for one patient reference, one service and one arrival window. It has its own lifecycle.
ClinicWindow	Entity / configuration	A clinic-defined interval for a service on a local service date, with capacity and a unique identifier.
QueueTicket	Entity / aggregate	A queue entry created only after accepted check-in. It has a separate lifecycle from the appointment.
AuditEvent	Entity / append-only record	A traceable record of a requested, accepted, rejected or corrected operation, including actor role and server timestamp.
TimeWindow	Value object	A pair of timezone-aware instants where start is strictly before end; interpreted as [start, end).
AppointmentState	Enumeration	CONFIRMED, CHECKED_IN, CANCELLED, NO_SHOW or COMPLETED.
TicketState	Enumeration	WAITING, CALLED, IN_CONSULTATION, FINISHED or WITHDRAWN.
Check-in request	Command	A request submitted by a kiosk or authorised staff; it is not itself proof that check-in succeeded.
CheckInAccepted	Domain event	A server-validated event that moves an eligible appointment into CHECKED_IN and permits ticket issuance.
No-show eligible	Derived decision	The time threshold has passed; the appointment still requires an authorised reconciliation decision before NO_SHOW is recorded.
Active ticket	Derived concept	A ticket in WAITING, CALLED or IN_CONSULTATION. FINISHED and WITHDRAWN tickets are not active.
Pseudonymous patient reference	Value / identifier	A reference that avoids putting a patient name or national ID in queue records. It can still be personal data if linkable.

2.2 Context map

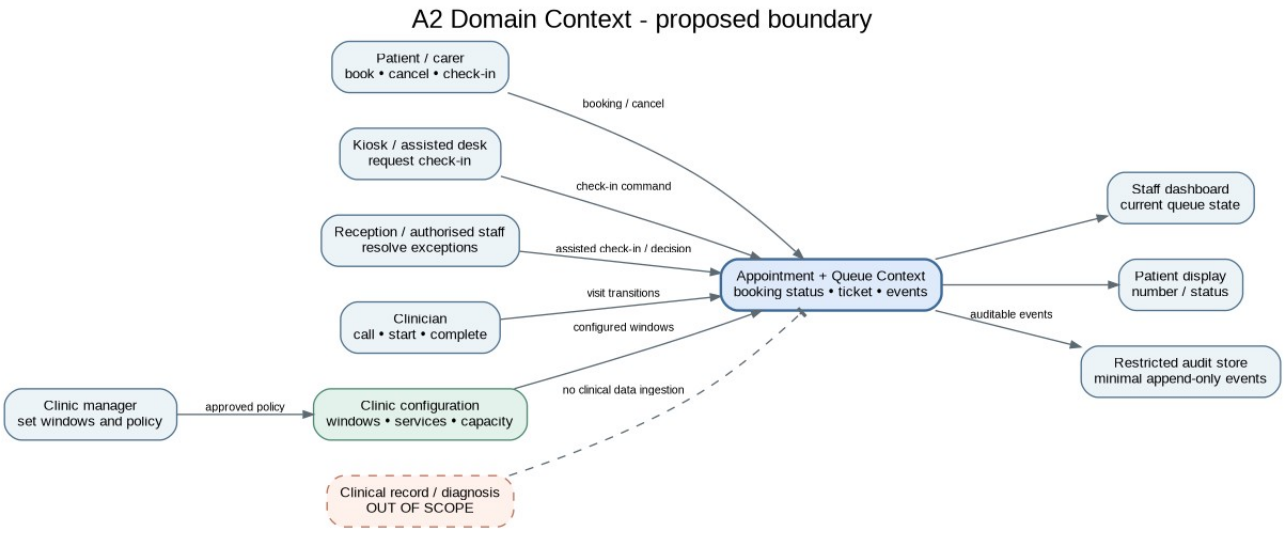


Figure 1. Proposed bounded context: booking, arrival confirmation, queue ticket and minimal audit events; clinical records are outside the boundary.

The patient or carer can request a booking or cancellation. The kiosk/assisted desk submits a check-in command, but the backend validates it and controls the state change. Clinic configuration supplies valid windows and capacity. Authorised staff handle exceptions. The staff dashboard and patient display are read-only views of approved queue state; they do not create their own source of truth.

2.3 Entity and value-object responsibilities

Object	Owns / decides	Must not own / decide
ClinicWindow	Service/date, start/end, capacity and window availability.	Patient identity, medical diagnosis or actual check-in status.
Appointment	Booking identity, patient reference, chosen window, appointment state and check-in/no-show timestamps.	Queue ordering or clinical urgency.
QueueTicket	Queue number, appointment reference, issue time and queue state.	Booking cancellation policy or diagnosis.
AuditEvent	Append-only trace of action, time, actor role, result and linked aggregate.	Password, access token or unrestricted free-text medical notes.
TimeWindow	Valid aware times and boundary interpretation.	Clinic opening policy or capacity decision.

3. Domain Events and Process Flow

A command asks the system to do something. A domain event records something that has already been accepted as having happened. Separating these concepts is important: a rejected check-in request must not emit CheckInAccepted and must not issue a queue ticket.

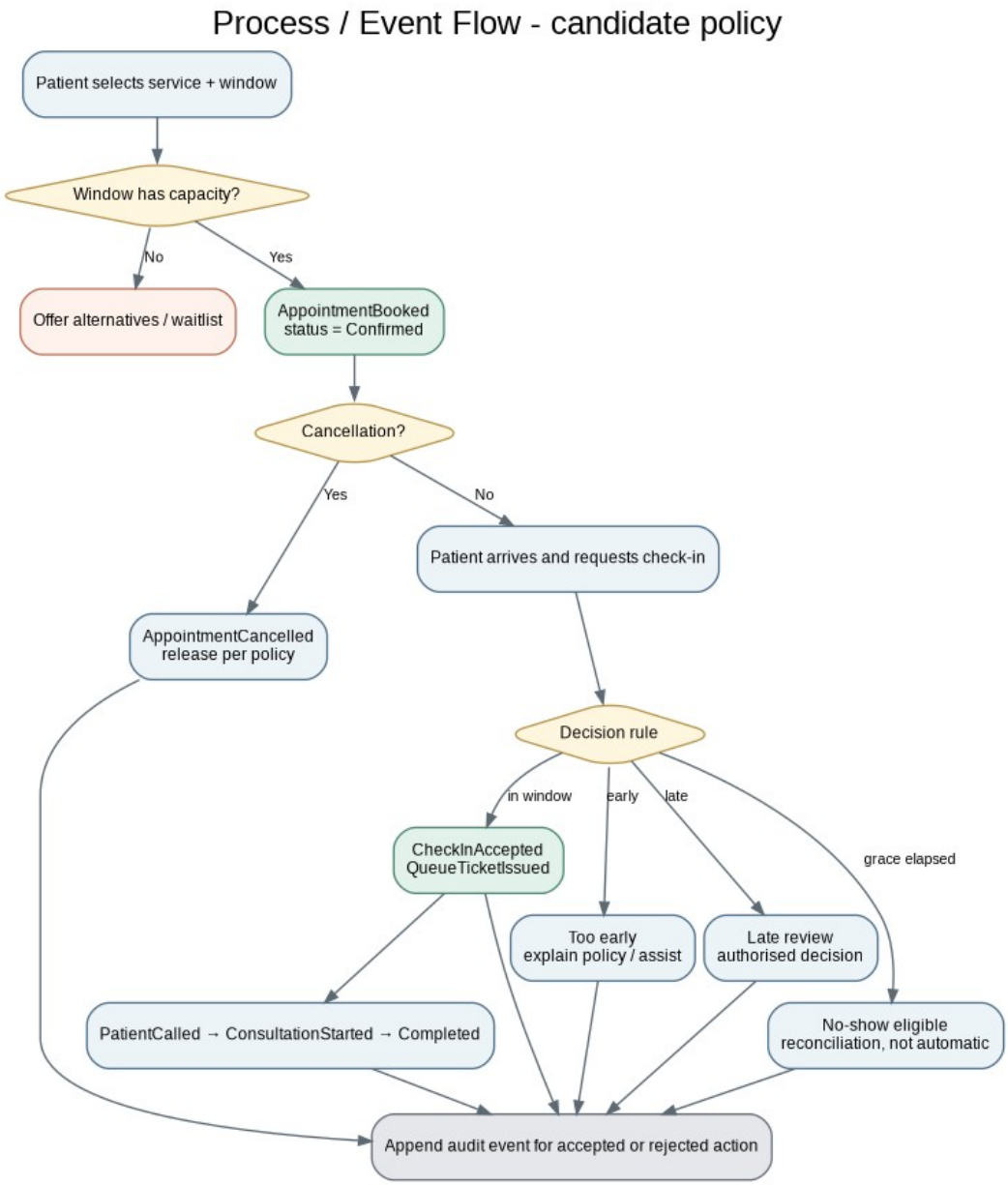


Figure 2. Candidate process/event flow. A no-show becomes eligible for reconciliation after the configured threshold; it is not automatic.

3.1 Event catalogue

Event / command	Origin / authority	Required effect or constraint
AppointmentBookingRequested	Patient / booking client	Validate service, window capacity and required fields; no confirmed state until accepted.
AppointmentBooked	Booking service	Create an immutable appointment ID and set CONFIRMED after successful validation.
AppointmentCancellationRequested	Patient or authorised staff	Validate that the appointment is still cancellable and actor is authorised.
AppointmentCancelled	Booking service	Set CANCELLED and apply the clinic-approved capacity-release rule.
CheckInRequested	Kiosk or assisted desk	Treat as an untrusted command; validate booking reference, state, time, identity/authentication and idempotency key.

Event / command	Origin / authority	Required effect or constraint
CheckInAccepted	Server / appointment service	Record server timestamp, change state to CHECKED_IN and emit a durable event once.
CheckInRejected	Server / appointment service	Record the reason category without exposing sensitive details; do not issue a ticket.
QueueTicketIssued	Queue service	Create one ticket linked to the checked-in appointment; unique ticket and queue number.
PatientCalled	Authorised clinic staff	Change an eligible ticket from WAITING to CALLED; record actor/time.
ConsultationStarted	Authorised clinic staff	Change ticket from CALLED to IN_CONSULTATION.
ConsultationCompleted	Authorised clinic staff / approved workflow	Change ticket to FINISHED and appointment to COMPLETED.
NoShowEligible	Derived from time and current state	Indicate that reconciliation may be needed; does not itself mutate appointment state.
NoShowRecorded	Authorised staff after reconciliation	Only record if no accepted check-in exists and the approved threshold has passed.
AppointmentExceptionCorrected	Authorised supervisor	Append a correction event with reason; do not erase the original audit history.

3.2 Normal process in sequence

1. The clinic publishes a service and valid arrival windows with capacity.
2. The patient selects a service and available window. The backend validates capacity and creates a CONFIRMED appointment.
3. The patient arrives in the allowed window and presents the approved booking reference at a kiosk or assisted desk.
4. The server evaluates the time window, appointment state, authorisation and idempotency. A valid request records CheckInAccepted.
5. The queue service creates exactly one active QueueTicket and writes an audit event in the same logical transaction or with a reliable outbox pattern.
6. Authorised staff call the patient. QueueTicket proceeds through the allowed queue states; appointment becomes COMPLETED after the visit is recorded complete.
7. If the threshold passes without accepted check-in, the system presents a reconciliation candidate. An authorised staff member decides whether to record NO_SHOW under local policy.

4. State Models

Appointment lifecycle and queue-ticket lifecycle are separate. This is a deliberate modelling choice: an appointment may be confirmed while no ticket exists, and a ticket describes the patient's position after arrival rather than the booking itself.

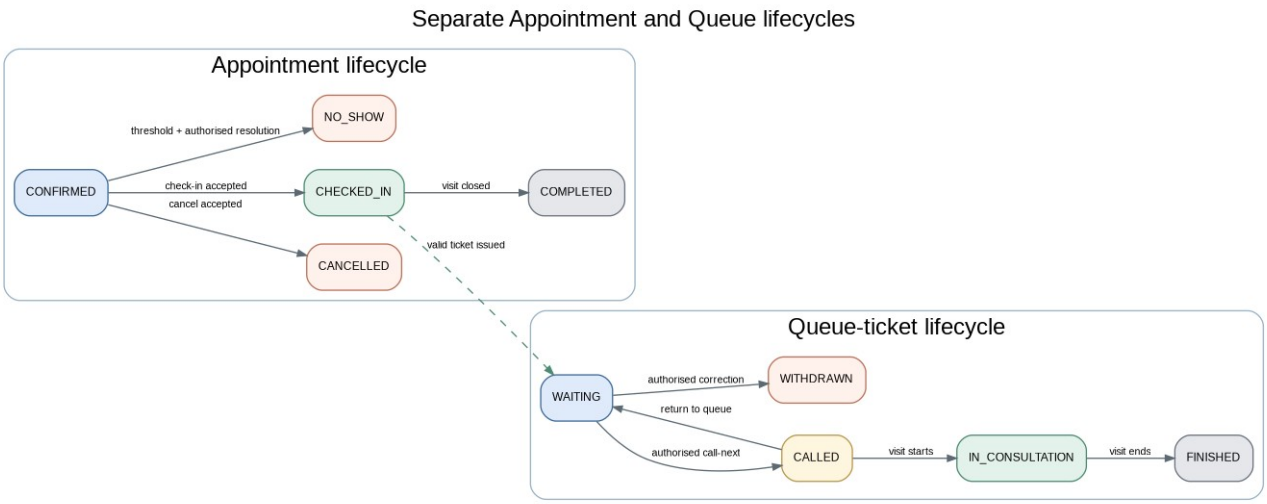


Figure 3. Separate appointment and queue-ticket lifecycles. A ticket is linked to accepted check-in; one status field must not try to represent both concepts.

4.1 Appointment transitions

From	Event / guard	To	Forbidden or constrained behaviour
—	Valid booking accepted	CONFIRMED	Cannot create a confirmed appointment without a valid window and capacity.
CONFIRMED	Check-in accepted in window	CHECKED_IN	The kiosk request alone is insufficient; server validation is required.
CONFIRMED	Cancellation accepted	CANCELLED	Cancellation must pass state and authorisation checks.
CONFIRMED	No-show reconciliation approved after threshold	NO_SHOW	Must not coexist with a prior accepted check-in; never automatic only because time passed.
CHECKED_IN	Consultation recorded complete	COMPLETED	Requires a check-in timestamp and a valid ticket/visit history.
CANCELLED / NO_SHOW / COMPLETED	Any ordinary check-in request	No transition	Terminal in the prototype. Corrections require a separately authorised, audited exception workflow.

4.2 Queue ticket transitions

From	Allowed target(s)	Guard / audit requirement
WAITING	CALLED, WITHDRAWN	Only authorised queue/clinic action can call or withdraw.
CALLED	IN_CONSULTATION, WAITING, WITHDRAWN	Return to WAITING requires an authorised recall/correction; every transition is audited.
IN_CONSULTATION	FINISHED	Completion is recorded by an authorised actor or approved integration.
FINISHED	None	Terminal in the normal workflow; corrections are not destructive edits.

From	Allowed target(s)	Guard / audit requirement
WITHDRAWN	None	Terminal in the normal workflow; new ticket requires a documented, authorised policy.

State-transition principle

A state change must be validated against the current persisted state, not only a state displayed by a client. Use an atomic transaction or optimistic concurrency check so two simultaneous requests cannot both issue active tickets.

5. Data Dictionary and Data Authority

Types and fields below are a proposed minimal schema for the coursework model. Names and formats are implementation choices, not a representation of any specific clinic's existing database. "Sensitive by association" means a value may become personal or sensitive when it can be linked to a person or medical visit.

5.1 ClinicWindow

Field	Type	Meaning	Authority / validation	Privacy / notes
window_id	string / UUID	Unique window identifier.	Clinic configuration owner; immutable after referenced.	Unique; non-empty. Operational metadata.
clinic_id	string	Clinic/service location identifier.	Approved configuration.	Must reference known clinic. Operational metadata.
service_code	enum/string	Non-diagnostic service category.	Clinic configuration allow-list.	Reject unknown codes; no diagnosis free text.
local_date	ISO date	Service date in clinic timezone.	Clinic configuration.	Valid date; interpreted with Asia/Shanghai.
window_start	aware datetime	Start instant for patient arrival.	Clinic config, normalized by server.	Must be aware; strictly before end.
window_end	aware datetime	End instant for arrival window.	Clinic config, normalized by server.	Must be aware; greater than start; excluded boundary.
capacity	positive integer	Maximum bookings allocated to window.	Clinic manager / scheduling authority.	Must be positive; concurrent booking check must be atomic.
status	enum	OPEN, FULL, CLOSED or CANCELLED configuration state.	Configuration service.	Allow-listed; changes audited.

5.2 Appointment

Field	Type	Meaning	Authority / validation	Privacy / notes
appointment_id	string / UUID	Unique reservation identifier.	Server-generated; immutable and unique.	Personal by association; no direct identity in ID.
patient_ref	pseudonymous string	Minimal reference to a patient record held by an approved source.	Clinic-approved identity/booking system; synthetic in this model.	Still personal if linkable; never use name/national ID here.
window_id	string / FK	Window assigned to the appointment.	Booking service.	Must reference an existing open/valid window.

Field	Type	Meaning	Authority / validation	Privacy / notes
state	enum	Appointment lifecycle state.	Server transition service only.	Enum; every change must be auditable.
created_at	aware datetime	Time booking was accepted.	Server clock.	Required, aware and immutable.
checked_in_at	nullable aware datetime	Accepted check-in time.	Server clock after validated check-in.	Null before check-in; required for CHECKED_IN/COMPLETED.
no_show_recorded_at	nullable aware datetime	Time an authorised no-show reconciliation is recorded.	Server after authorised staff action.	Only for NO_SHOW and after end + approved grace.
cancellation_reason_code	nullable enum	Small controlled category for cancellation if needed.	Patient/authorised staff request via allow-list.	Avoid free text; collect only if necessary.

5.3 QueueTicket

Field	Type	Meaning	Authority / validation	Privacy / notes
ticket_id	string / UUID	Unique queue ticket identity.	Server-generated; unique.	Operational, potentially linkable.
appointment_id	string / FK	Appointment that created the ticket.	Server-side transaction after accepted check-in.	Must reference eligible checked-in/completed appointment.
queue_number	string	Human-readable queue number.	Server-side allocator.	Unique within a queue session; not client-controlled.
session_id	string / FK	Queue session or clinic/day identifier.	Clinic configuration / queue service.	Used for queue-number uniqueness scope.
state	enum	WAITING, CALLED, IN_CONSULTATION, FINISHED, WITHDRAWN.	Server transition service.	Only allowed transitions; every change audited.
issued_at	aware datetime	Ticket issuance time.	Server clock.	Cannot precede accepted check-in; timezone-aware.
withdrawn_reason_code	nullable enum	Controlled reason if ticket is withdrawn.	Authorised staff or approved workflow.	No unnecessary narrative patient details.

5.4 AuditEvent

Field	Type	Meaning	Authority / validation	Privacy / notes
event_id	string / UUID	Unique audit event identity.	Server-generated; unique; append-only.	Security/operational metadata.
aggregate_id	string / FK	Appointment or ticket related to the event.	Server event writer.	Must reference a known object or rejected command record.
event_type	enum/string	Accepted, rejected, transitioned or corrected action.	Allow-list in the server.	No free-text health notes or credentials.
occurred_at	aware datetime	Server event time.	Trusted server clock.	Timezone-aware; ordering rules defined per aggregate.
actor_role	enum	Patient, kiosk service, receptionist, clinician, supervisor or system.	Authenticated session / service identity.	Record role, not password or session token.

Field	Type	Meaning	Authority / validation	Privacy / notes
outcome_code	enum	Success, rejection or exception category.	Server.	Minimal reason code; restrict who can read audit.

5.5 Data quality checks

- Identifiers are generated on the server, non-empty and unique; foreign keys resolve to the intended record.
- Statuses and reason codes use enumerations/allow-lists; unknown values are rejected rather than interpreted permissively.
- All timestamps include timezone information; start < end; event ordering and accepted check-in timestamps are consistent.
- Capacity checks and ticket issuance are atomic to prevent simultaneous bookings or duplicate tickets.
- Repeated requests use an idempotency key or return the already-created outcome; retries must not duplicate side effects.
- Rejected attempts are traceable using minimal, non-sensitive reason codes; client-supplied timestamps and queue numbers are not trusted.

6. Invariants, Violating Examples and Automated Checks

An invariant is a condition that must hold for every valid saved state. The examples below include both positive records and deliberate violations. Automated checks use the Python reference model in Appendix B and synthetic fixtures only.

Invariant	Rule that must hold	Violating example	Automated check
I1 — Valid time window	Each ClinicWindow has timezone-aware start and end timestamps, with start < end.	Start=09:00, end=09:00; reversed time; or a naive timestamp with no timezone.	TimeWindow.is_valid() and contains(); tests cover valid, zero-length, reversed and naive windows.
I2 — Appointment/time coherence	CHECKED_IN and COMPLETED require checked_in_at; CONFIRMED, CANCELLED and NO_SHOW must not have checked_in_at. NO_SHOW requires a recorded time at or after end + grace.	State=CHECKED_IN but checked_in_at=null; or NO_SHOW while checked_in_at=09:30; or no-show recorded before the threshold.	appointment_violations(); tests cover coherent and incoherent status/timestamp combinations.
I3 — At most one active queue ticket	For any appointment_id, no more than one ticket may be in WAITING, CALLED or IN_CONSULTATION. Ticket IDs and queue number per session are unique.	Appointment A-1 has active tickets T-1 (WAITING) and T-2 (CALLED); two tickets in same session share Q-001.	duplicate_active_ticket_appointments() and ticket_registry_violations(); duplicate cases are asserted in tests.
I4 — Ticket requires accepted check-in	A ticket can only be issued if the appointment is CHECKED_IN, has a valid checked_in_at, and has no existing active ticket.	A confirmed booking with no check-in receives a ticket; a retry after ticket creation creates a second ticket.	can_issue_ticket(); tests assert pre-check-in issuance is false and one-time issuance becomes false once a ticket exists.
I5 — No-show needs authorised reconciliation	A NO_SHOW decision requires state CONFIRMED, no accepted check-in, now >= end + grace and an authorised staff role. Passing time alone does not change state.	Patient role marks themselves NO_SHOW; a receptionist marks NO_SHOW 14 minutes after the 11:00 end when grace=15; checked-in appointment is marked NO_SHOW.	can_record_no_show() and appointment_violations(); tests check role, threshold and check-in conflicts.
I6 — Queue states follow allowed transitions	Ticket transitions must follow the explicit state graph; FINISHED and WITHDRAWN are terminal in normal flow.	FINISHED -> WAITING without a correction workflow; WITHDRAWN -> CALLED.	ticket_transition_allowed(); tests cover allowed transitions and terminal-state violations.

6.1 How the tests challenge the model

The tests do not prove the system safe for clinical deployment. They prove only that the reference implementation returns the specified outcomes for tested examples. The suite deliberately includes counterexamples: zero-length/reversed windows, timezone-naive times, duplicate active tickets, ticket creation before check-in, no-show attempts before the grace threshold or by an unauthorised actor, and invalid transitions from terminal queue states.

Concurrency risk that unit tests do not resolve

In production, two kiosks could request check-in at nearly the same time. A database uniqueness constraint plus an atomic transaction or idempotency mechanism is needed; a pure Python unit test alone cannot establish concurrency safety.

7. Material Rule: Check-In Decision Table and Contract

The material rule is whether a check-in request may be accepted, must be routed for staff review, or must be rejected. The table assumes a valid configured window [start, end) and a prototype grace parameter of 15 minutes. A clinic must approve the actual grace and exception policy before deployment.

Rule	State	Time / condition	Decision	System effect
D1	Cancelled	Any time	REJECT_CANCELLED	Do not issue ticket; provide rebooking/assisted information.
D2	Completed or NO_SHOW	Any time	REJECT_CLOSED	Do not reopen automatically; authorised correction is a separate audited flow.
D3	CHECKED_IN	Any time	ALREADY_CHECKED_IN	Return existing ticket/current queue status; do not create another ticket.
D4	CONFIRMED	now < start	TOO_EARLY	Explain window; no ticket issued.
D5	CONFIRMED	start <= now < end	ACCEPT_CHECK_IN	Record server check-in time and issue one ticket atomically.
D6	CONFIRMED	end <= now < end + grace	LATE_REVIEW	Escalate to authorised staff; no automatic rejection or ticket.
D7	CONFIRMED	now >= end + grace	NO_SHOW_ELIGIBLE	Offer to authorised staff for reconciliation; do not automatically set NO_SHOW.
D8	Any	Invalid window/time or unauthenticated request	REJECT_INVALID	Do not mutate appointment or ticket; log a minimal rejection code.

7.1 Preconditions and postconditions

Type	Contract
Preconditions	Window is valid; now is timezone-aware; grace_minutes >= 0; request is associated with a known appointment; actor/request passes authentication and authorisation. The pure decision function validates window/time/grace; production service must also validate identity and transaction state.
Postcondition: in window	For CONFIRMED with start <= now < end, decision is ACCEPT_CHECK_IN. After a successful transaction, appointment is CHECKED_IN, checked_in_at is set by server, and exactly one active ticket exists.
Postcondition: repeat check-in	For CHECKED_IN, return ALREADY_CHECKED_IN and the existing ticket/status; no new ticket or second accepted event is generated.
Postcondition: late/no-show	At end, decision is LATE_REVIEW. At end + grace, decision becomes NO_SHOW_ELIGIBLE. Neither outcome by itself changes state to NO_SHOW.
Side-effect rule	The decision function is pure: it chooses an outcome and does not write a database, issue a ticket or mark a patient absent. Persisted changes require server validation and audit.

7.2 Formal assertions

- I1: aware(start) AND aware(end) AND start < end.
- I2: state in {CHECKED_IN, COMPLETED} => checked_in_at != null; state in {CONFIRMED, CANCELLED, NO_SHOW} => checked_in_at == null.
- I3: for every appointment a, count(active tickets for a) <= 1.
- I4: create_ticket(a) => a.state == CHECKED_IN AND a.checked_in_at != null AND count(active tickets for a) == 0.
- I5: record_no_show(a, now, role) => a.state == CONFIRMED AND a.checked_in_at == null AND now >= end + grace AND role is authorised.
- I6: next_ticket_state must belong to AllowedTransitions[current_ticket_state].

8. Data Source, Authority, Quality, Retention, Deletion and Privacy

8.1 Data source and authority

Data/action	Authoritative source	Trust boundary
Clinic, service, time window and capacity	Clinic-approved configuration managed by an authorised clinic role.	Client/kiosk cannot create or change available windows or capacity directly.
Booking request	Patient/carer client provides requested service/window; server checks availability and validates fields.	A submitted request is not a confirmed booking until backend acceptance.
Accepted check-in time and state	Server clock and appointment transition service after validation.	Do not trust a timestamp or status supplied by kiosk/browser.
Ticket ID and queue number	Queue service / server allocator.	Never accept client-selected queue number; unique per session.
No-show decision	Authorised staff reconciliation using server time and the approved clinic policy.	Time passing only creates eligibility; UI or AI must not automatically label a patient absent.

Data/action	Authoritative source	Trust boundary
Audit event	Server event writer linked to authenticated actor/service identity.	Append-only; restricted read access; no passwords, tokens or clinical narratives.

8.2 Data quality and integrity

The server is the source of truth for state transitions, while a clinic-approved configuration is the source of truth for service windows and capacity. Every accepted action must pass required-field, enum, reference, timestamp, authorisation and current-state validation. Capacity reservation and queue-ticket creation must be atomic or protected by a unique constraint to address concurrent requests. The service should use idempotency keys or equivalent deduplication so network retries do not create duplicate appointments or tickets.

8.3 Retention and deletion

Data category	Coursework prototype	Future clinic deployment
Synthetic appointments/tickets	Keep only as long as needed to reproduce tests; reset/remove generated outputs when no longer needed.	Clinic must approve a documented schedule based on applicable law, operational need and institutional recordkeeping rules.
Audit events	Synthetic, append-only within a test run; no secrets or real personal data.	Define purpose, retention duration, access and legal-hold handling. Avoid keeping logs indefinitely by default.
Exports, notebooks and backups	Use synthetic values only; generated results can be deleted and recreated from fixed code.	Deletion policy must cover exports and backups where feasible; document any retained copies required by law.
Real identity and clinical records	Not collected and not included in repository.	Remain in approved clinic systems. Any linkage or access requires a separately approved purpose, access model and retention process.

8.4 Privacy and security boundary

- The PRC Personal Information Protection Law identifies medical-health information as sensitive personal information and requires a specific purpose, necessity and strict protection measures; sensitive-information processing has additional notice/consent rules subject to applicable legal provisions [3]. This report is not legal advice.
- Use the minimum patient reference required to coordinate booking and arrival. A pseudonymous reference can still be personal information when the person can be reidentified through another system.
- Do not store diagnoses, symptoms, clinical notes, national identity card numbers, passwords, API keys, session tokens or payment data in this queue-coordination model.
- Authenticate staff and services; apply role-based authorisation; audit privileged changes; encrypt communications and sensitive storage; restrict audit access; and avoid sensitive values in logs.
- Provide an assisted check-in and approved downtime procedure so a kiosk/network failure or lack of smartphone does not silently erase a valid appointment. Reconciliation after downtime must prevent duplicate tickets.
- The medical and health institutions cybersecurity management measures call for institutional responsibility, security management and lifecycle controls for health data [4]. A production implementation needs local security/compliance review and cannot claim compliance from this specification alone.

9. Scenario Walkthrough Record

Walkthrough type and limitation

A desk-based model walkthrough was performed on 12 October 2026 using the explicit synthetic scenarios below and the reference test suite. This is a specification-level walkthrough, not an interview, clinic observation, stakeholder review, user study or clinical validation. No external reviewer is claimed.

ID / scenario	Input	Expected decision / state	Rule(s)	Walkthrough observation
MW-01 Normal arrival	CONFIRMED; window 09:00–11:00; request at 10:05.	ACCEPT_CHECK_IN; set CHECKED_IN using server time; issue one active ticket.	I1, I2, I3, I4	Consistent with the proposed model; covered by accepted-in-window and ticket-issue tests.
MW-02 Exact start boundary	CONFIRMED; request at 09:00 exactly.	Accept, because the start boundary is inclusive.	I1, decision contract	Boundary convention is explicit and automated.
MW-03 Exact end boundary	CONFIRMED; request at 11:00 exactly.	LATE_REVIEW, because end is exclusive; no automatic ticket.	I1, I5	A different clinic policy could choose another rule; approve before deployment.
MW-04 Grace threshold	CONFIRMED; no check-in; request/time at 11:15 where grace=15 minutes.	NO_SHOW_ELIGIBLE; staff reconciliation is required. Do not automatically set NO_SHOW.	I2, I5	The code enforces the threshold decision; actual grace duration remains a clinic policy choice.
MW-05 Cancelled booking	Appointment is CANCELLED; patient presents the old reference.	Reject check-in; no new ticket.	I4, allowed states	Covered by decision-table test.
MW-06 Duplicate request	Appointment is already CHECKED_IN and one active ticket exists; kiosk retries.	Return existing ticket/status; do not create another ticket.	I3, I4	Decision is idempotent in the model; production transaction and retry behaviour need integration tests.
MW-07 Duplicate active tickets	One appointment has T-1 WAITING and T-2 CALLED.	Treat as invalid state and reject/fail the invariant check.	I3	Automated test identifies the duplicate appointment ID.
MW-08 Contradictory no-show	Appointment has checked_in_at=09:30 but state is NO_SHOW at 11:20.	Invalid; accepted check-in and NO_SHOW cannot coexist.	I2, I5	Automated coherence check detects it.
MW-09 Kiosk/network outage	Patient arrives but the kiosk cannot contact the server.	Use only the clinic-approved assisted/downtime route; do not trust an offline client to silently set final state. Reconcile once online.	Authority, audit, accessibility	Operational fallback must be validated with a real clinic; it is a boundary requirement, not implemented by this unit model.
MW-10 No smartphone	Patient has a valid booking but cannot use the digital kiosk.	Provide staff-assisted confirmation under the same server-side rules.	Accessibility, identity boundary	Required workflow alternative; no patient test is claimed.

9.1 Ambiguities surfaced and modelling decisions

Question	Decision adopted in this specification	Status
Does exact window end count as on time?	No. The window is [start, end); start is inclusive, end is excluded.	Explicit prototype rule; clinic approval needed.
Does the system automatically mark no-show?	No. Time creates NO_SHOW_ELIGIBLE; authorised staff reconcile.	Chosen to avoid automatic exclusion without context.

Question	Decision adopted in this specification	Status
Can a booking produce more than one active ticket?	No. At most one ticket can be WAITING, CALLED or IN_CONSULTATION for an appointment.	Implemented and tested in reference model.
Can check-in state come from kiosk data?	No. Kiosk submits a command; server decides and timestamps.	System authority boundary.
What about clinical urgency?	Clinical triage/priority is outside this model; follow the clinic's existing clinical process.	Scope boundary requiring operational alignment.
What is the exact retention period?	Not assigned a fabricated number; clinic/legal owners must approve the schedule.	Unresolved for production; prototype uses synthetic data only.

10. Verification Results, Limits and AI-Use Log

10.1 Reproducibility record

Item	Recorded value
Reference implementation	Python standard-library model; source listing in Appendix B.
Runtime	Python 3.13.5
Randomness / seed	No random generation used; no seed required.
Test data	Deterministic synthetic fixtures embedded in unit tests; no external dataset.
Command	python -m unittest -v test_clinic_model
Recorded result	36 tests run; 36 passed; 0 failures; exit code 0.
What this means	The coded rules passed this unit suite; it does not prove user acceptance, clinical effectiveness, security certification or concurrency safety.

10.2 Limits and evidence that could challenge the design

- No target clinic was selected, observed or interviewed. The states and responsibilities are a proposed model, not a verified representation of all Chinese clinics.
- A flexible arrival window could concentrate arrivals at its beginning or near its end. It does not, by itself, reduce actual waiting time; capacity and arrival distribution must later be measured.
- An appointment no-show may leave capacity unused, but the proposed workflow does not recover it automatically. A wait-list or release policy is deliberately out of scope until clinic rules and fairness consequences are understood.
- A 15-minute grace period could be unfair or operationally unsuitable in some settings. It is configurable and should be validated, not hard-coded as a universal rule.
- A kiosk introduces accessibility, outage and identity-verification risks. Assisted check-in and an audit-friendly fallback are required for the design to be credible.
- Published studies from Jinan or Guangzhou hospital settings provide context only; their rates or results are not the baseline of the intended clinic [1,2].
- The unit suite does not test database transactions, multi-kiosk concurrency, authentication integration, usability, accessibility, privacy controls, or deployment-level retention/deletion.

10.3 AI-use and source-verification log

AI/tool use	Advice/content	Treatment	Verification and limits
OpenAI ChatGPT	Interpret A2 instructions; draft glossary, process, state, data dictionary, invariants, decision table and test scenarios.	Accepted as a proposed model for the same A1 clinic project.	Rules are design choices, not clinic facts. Executable model was run against synthetic tests.
OpenAI ChatGPT	Generate reference Python functions and unit-test cases for boundary and violating examples.	Accepted for this specification appendix.	Ran the resulting source in Python 3.13.5; recorded 36 passing tests. Tests do not establish clinical or production correctness.
Public web sources	Find research and primary/official material on Chinese outpatient appointments, PIPL and medical-sector cybersecurity.	Used as background/context, not as local operational evidence.	Opened listed pages and checked their titles and relevant claims. Specific clinic workflow, policy applicability and production compliance remain unverified.
Suggestions deliberately constrained/rejected	Infer a universal no-show rate, fixed grace period, retention duration, or claim stakeholder/clinic validation.	Not accepted. Grace=15 minutes is only a test parameter; no retention period is invented; no clinic review is claimed.	Keeps unknown real-world facts visible instead of fabricating data, source claims or user research.

10.4 Source-verification log

Source	Claim used	Verification / limitation
[1] JMIR outpatient no-show study	A published Jinan hospital dataset reported 48,777 online bookings and a 15% no-show rate.	Number is attributed only to that hospital and period; not reused as the target clinic baseline.
[2] BMC reservation-service study	A Chinese tertiary-hospital study evaluated a comprehensive reservation service and outcomes including waiting time and satisfaction.	Shows appointment-workflow design has been studied; does not prove this proposed two-stage design works.
[3] Personal Information Protection Law	Medical-health information is sensitive personal information; specific-purpose/necessity and strict safeguards are required, with additional provisions for sensitive data.	Used for privacy boundary; legal interpretation and applicability require qualified/local review.
[4] Medical and health institution cybersecurity measures	Institutions have responsibility for cybersecurity and lifecycle protection of health data.	Used as a high-level security constraint; no certification of compliance is claimed.
[5] State Council “Internet Plus Healthcare” guideline	National policy encourages internet-enabled and integrated healthcare services.	Policy context only; it does not require or endorse this specific system design.

11. Conclusion

The project now has a consistent proposed domain vocabulary, separate appointment and ticket lifecycles, explicit data fields and authorities, six invariants with violating examples, a check-in decision table and an executable reference model. The 36-test run provides repeatable evidence that the implemented rules behave as specified for the selected synthetic cases. It does not establish that the model reflects a particular clinic’s real policy or that the system will reduce waiting times or no-shows. The next engineering gate should use an authorised clinic workflow walkthrough to confirm terminology, capacity rules, grace/late-arrival policy, staff roles, offline handling and retention/deletion requirements before architecture or implementation is treated as clinic-ready.

References

[1] "Who Misses Appointments Made Online? Retrospective Analysis of the Outpatient Department of a General Hospital in Jinan, Shandong Province, China." Journal of Medical Internet Research (2020). Online appointment records from September 2017–February 2018; 48,777 records and 15% no-show rate in that setting. https://pmc.ncbi.nlm.nih.gov/articles/PMC7708679/

[2] Xie, W. et al. "Effects of a comprehensive reservation service for non-emergency registration on appointment registration rate, patient waiting time, patient satisfaction and outpatient volume in a tertiary hospital in China." BMC Health Services Research 19, 782 (2019). DOI: 10.1186/s12913-019-4652-6. https://pmc.ncbi.nlm.nih.gov/articles/PMC6824126/

[3] Personal Information Protection Law of the People's Republic of China, Articles 28–30. English-language text hosted by the National People's Congress website. https://en.npc.gov.cn.cdurl.cn/2021-12/29/c_694559_2.htm

[4] National Health Commission, National Administration of Traditional Chinese Medicine and National Administration of Disease Control and Prevention. Measures on Cybersecurity Management for Medical and Health Institutions (2022), official government publication. https://app.www.gov.cn/govdata/gov/202208/31/488953/article.html

[5] General Office of the State Council of the People's Republic of China. Guidance on Promoting the Development of "Internet Plus Healthcare" (2018), official policy publication. https://english.www.gov.cn/policies/latest_releases/2018/04/28/content_281476127312948.htm

Accessed: 12 October 2026.

Appendix A. Reproduction Instructions and Test Output

The reference model and unit tests are also printed in Appendix B so the essential specification code is contained in this single document. To reproduce the recorded checks, copy Appendix B into files named clinic_model.py and test_clinic_model.py in the same directory and run the command below with Python 3.11 or newer. The recorded run used Python 3.13.5 and the Python standard library only.

Command

python -m unittest -v test_clinic_model

Recorded output summary

Metric	Result
Test cases executed	36
Passed	36
Failed / errors	0
Elapsed time	0.001 seconds in recorded run (machine-dependent)
Exit status	0

Test output (recorded)

```
----------------------------------------------------------------------
Ran 36 tests in 0.001s

OK
```

Coverage by rule

Test group	Coverage
I1 — TimeWindow	Valid window, zero-length and reversed windows, naive timestamp, start-inclusive/end-exclusive interval.
Decision table	Too early, exact start, in-window, exact end, just before threshold, threshold, cancelled, repeated check-in, completed, invalid time and negative grace.
I2 — Appointment/time coherence	Confirmed valid, checked-in without/with timestamp, contradictory no-show with check-in, early no-show, valid post-threshold no-show.
I3/I4 — Tickets	Duplicate active tickets, withdrawn ticket, issuance only after check-in, one-time issuance and queue-number uniqueness.
I5 — No-show and queue transitions	Staff role, threshold, check-in conflict, allowed transition and terminal-state rejection.

No real clinic dataset was used. No random generation was used, so a seed is not applicable. The synthetic fixture dates are fixed in the tests for deterministic boundary checks. If tests fail in a later environment, preserve the failure output and update the model/test together rather than editing the output to imply success.

Appendix B. Complete Reference Model and Automated Tests

This source listing is a small executable specification, not production software. It contains no real patient data, credentials, external services or hidden dataset. The code is included here to make the core rules inspectable in a single file.

B1. clinic_model.py

```
"""Reference specification for the ISE-A2 clinic appointment workflow.

Uses synthetic/test values only. It is not production clinical software.
"""
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timedelta
from enum import Enum
from collections import Counter


class AppointmentState(str, Enum):
    CONFIRMED = "CONFIRMED"
    CHECKED_IN = "CHECKED_IN"
    CANCELLED = "CANCELLED"
    NO_SHOW = "NO_SHOW"
    COMPLETED = "COMPLETED"


class TicketState(str, Enum):
    WAITING = "WAITING"
    CALLED = "CALLED"
    IN_CONSULTATION = "IN_CONSULTATION"
    FINISHED = "FINISHED"
    WITHDRAWN = "WITHDRAWN"


class CheckInDecision(str, Enum):
    TOO_EARLY = "TOO_EARLY"
    ACCEPT_CHECK_IN = "ACCEPT_CHECK_IN"
    LATE_REVIEW = "LATE_REVIEW"
    NO_SHOW_ELIGIBLE = "NO_SHOW_ELIGIBLE"
    REJECT_CANCELLED = "REJECT_CANCELLED"
    ALREADY_CHECKED_IN = "ALREADY_CHECKED_IN"
    REJECT_CLOSED = "REJECT_CLOSED"


ACTIVE_TICKET_STATES = {TicketState.WAITING, TicketState.CALLED, TicketState.IN_CONSULTATION}
ALLOWED_NO_SHOW_ROLES = {"receptionist", "queue_supervisor", "clinic_manager"}
GRACE_MINUTES_DEFAULT = 15  # Explicit prototype assumption; clinic must configure/approve.


def is_aware(value: datetime | None) -> bool:
    return value is not None and value.tzinfo is not None and value.utcoffset() is not None


@dataclass(frozen=True)
class TimeWindow:
    start: datetime
    end: datetime

    def is_valid(self) -> bool:
        return is_aware(self.start) and is_aware(self.end) and self.start < self.end

    def contains(self, instant: datetime) -> bool:
        if not self.is_valid():
            raise ValueError("Invalid appointment time window")
        if not is_aware(instant):
            raise ValueError("The timestamp must be timezone-aware")
        return self.start <= instant < self.end  # Half-open [start, end).


@dataclass(frozen=True)
class Appointment:
    appointment_id: str
    state: AppointmentState
    window: TimeWindow
    created_at: datetime
    checked_in_at: datetime | None = None
    no_show_recorded_at: datetime | None = None
```

```
@dataclass(frozen=True)
class QueueTicket:
    ticket_id: str
    appointment_id: str
    queue_number: str
    state: TicketState
    issued_at: datetime
    session_id: str

    @property
    def is_active(self) -> bool:
        return self.state in ACTIVE_TICKET_STATES


def appointment_violations(a: Appointment, grace_minutes: int = GRACE_MINUTES_DEFAULT) -> list[str]:
    """Return all detectable violations of appointment state/time coherence."""
    issues: list[str] = []
    if not a.appointment_id.strip():
        issues.append("appointment_id must be non-empty")
    if not a.window.is_valid():
        issues.append("window must have aware timestamps and positive duration")
    if not is_aware(a.created_at):
        issues.append("created_at must be timezone-aware")
    elif is_aware(a.checked_in_at) and a.checked_in_at < a.created_at:
        issues.append("checked_in_at cannot precede created_at")
    if a.state in {AppointmentState.CHECKED_IN, AppointmentState.COMPLETED} and not is_aware(a.checked_in_at):
        issues.append("CHECKED_IN/COMPLETED requires checked_in_at")
    if a.state in {AppointmentState.CONFIRMED, AppointmentState.CANCELLED, AppointmentState.NO_SHOW} and a.checked_in_at is not None:
        issues.append("CONFIRMED/CANCELLED/NO_SHOW cannot have checked_in_at")
    if a.state is AppointmentState.NO_SHOW:
        if not is_aware(a.no_show_recorded_at):
            issues.append("NO_SHOW requires an aware no_show_recorded_at")
        elif a.window.is_valid() and a.no_show_recorded_at < a.window.end + timedelta(minutes=grace_minutes):
            issues.append("NO_SHOW cannot be recorded before window end plus grace")
    if a.state is not AppointmentState.NO_SHOW and a.no_show_recorded_at is not None:
        issues.append("no_show_recorded_at is only valid for NO_SHOW")
    return issues


def duplicate_active_ticket_appointments(tickets: list[QueueTicket]) -> set[str]:
    counts = Counter(t.appointment_id for t in tickets if t.is_active)
    return {appointment_id for appointment_id, count in counts.items() if count > 1}


def ticket_registry_violations(tickets: list[QueueTicket]) -> list[str]:
    issues: list[str] = []
    ticket_ids = [t.ticket_id for t in tickets]
    if len(ticket_ids) != len(set(ticket_ids)):
        issues.append("ticket_id must be unique")
    if any(not t.ticket_id.strip() or not t.appointment_id.strip() for t in tickets):
        issues.append("ticket_id and appointment_id must be non-empty")
    if duplicate_active_ticket_appointments(tickets):
        issues.append("at most one active ticket is permitted per appointment")
    queue_keys = [(t.session_id, t.queue_number) for t in tickets]
    if len(queue_keys) != len(set(queue_keys)):
        issues.append("queue_number must be unique within a session")
    if any(not is_aware(t.issued_at) for t in tickets):
        issues.append("issued_at must be timezone-aware")
    return issues


def can_issue_ticket(a: Appointment, existing: list[QueueTicket]) -> bool:
    """Ticket creation is permitted only after recorded check-in and with no active ticket."""
    return (
        a.state is AppointmentState.CHECKED_IN
        and is_aware(a.checked_in_at)
        and not appointment_violations(a)
        and a.appointment_id not in duplicate_active_ticket_appointments(existing)
        and not any(t.appointment_id == a.appointment_id and t.is_active for t in existing)
    )


def decide_check_in(
    window: TimeWindow,
    state: AppointmentState,
    now: datetime,
    grace_minutes: int = GRACE_MINUTES_DEFAULT,
) -> CheckInDecision:
    """Pure decision function. It does not mutate appointments or issue tickets."""
    if not window.is_valid():
        raise ValueError("Invalid appointment time window")
    if not is_aware(now):
```

```
        raise ValueError("Current time must be timezone-aware")
    if grace_minutes < 0:
        raise ValueError("grace_minutes must be >= 0")
    if state is AppointmentState.CANCELLED:
        return CheckInDecision.REJECT_CANCELLED
    if state is AppointmentState.CHECKED_IN:
        return CheckInDecision.ALREADY_CHECKED_IN
    if state in {AppointmentState.NO_SHOW, AppointmentState.COMPLETED}:
        return CheckInDecision.REJECT_CLOSED
    if now < window.start:
        return CheckInDecision.TOO_EARLY
    if now < window.end:
        return CheckInDecision.ACCEPT_CHECK_IN
    if now < window.end + timedelta(minutes=grace_minutes):
        return CheckInDecision.LATE_REVIEW
    return CheckInDecision.NO_SHOW_ELIGIBLE


def can_record_no_show(
    a: Appointment,
    now: datetime,
    actor_role: str,
    grace_minutes: int = GRACE_MINUTES_DEFAULT,
) -> bool:
    """No-show is a staff-authorised reconciliation, never an automatic UI side effect."""
    if not a.window.is_valid() or not is_aware(now) or grace_minutes < 0:
        return False
    return (
        a.state is AppointmentState.CONFIRMED
        and a.checked_in_at is None
        and now >= a.window.end + timedelta(minutes=grace_minutes)
        and actor_role in ALLOWED_NO_SHOW_ROLES
        and not appointment_violations(a, grace_minutes)
    )


_ALLOWED_TICKET_TRANSITIONS = {
    TicketState.WAITING: {TicketState.CALLED, TicketState.WITHDRAWN},
    TicketState.CALLED: {TicketState.IN_CONSULTATION, TicketState.WAITING, TicketState.WITHDRAWN},
    TicketState.IN_CONSULTATION: {TicketState.FINISHED},
    TicketState.FINISHED: set(),
    TicketState.WITHDRAWN: set(),
}


def ticket_transition_allowed(current: TicketState, target: TicketState) -> bool:
    return target in _ALLOWED_TICKET_TRANSITIONS[current]
```

B2. test_clinic_model.py

```
import unittest
from datetime import datetime, timedelta, timezone
from clinic_model import (
    Appointment, AppointmentState, CheckInDecision, QueueTicket, TicketState,
    TimeWindow, appointment_violations, can_issue_ticket, can_record_no_show,
    decide_check_in, duplicate_active_ticket_appointments,
    ticket_registry_violations, ticket_transition_allowed,
)

CN = timezone(timedelta(hours=8))
def at(h, m=0): return datetime(2026, 10, 12, h, m, tzinfo=CN)
def window(): return TimeWindow(at(9), at(11))
def appointment(state=AppointmentState.CONFIRMED, checked=None, no_show=None):
    return Appointment("A-001", state, window(), at(8), checked, no_show)
def ticket(ticket_id="T-1", appointment_id="A-001", state=TicketState.WAITING, number="Q-001", session="S-1"):
    return QueueTicket(ticket_id, appointment_id, number, state, at(9, 10), session)

class TestInvariantI1Window(unittest.TestCase):
    def test_valid_positive_window(self): self.assertTrue(window().is_valid())
    def test_zero_length_window_rejected(self): self.assertFalse(TimeWindow(at(9), at(9)).is_valid())
    def test_reversed_window_rejected(self): self.assertFalse(TimeWindow(at(11), at(9)).is_valid())
    def test_naive_timestamp_rejected(self): self.assertFalse(TimeWindow(datetime(2026,10,12,9), at(11)).is_valid())
    def test_contains_uses_start_inclusive_end_exclusive(self):
        self.assertTrue(window().contains(at(9)))
        self.assertFalse(window().contains(at(11)))

class TestCheckInDecisionTable(unittest.TestCase):
    def test_too_early(self): self.assertEqual(decide_check_in(window(), AppointmentState.CONFIRMED, at(8,59)), CheckInDecision.TOO_EARLY)
    def test_start_boundary_accepted(self): self.assertEqual(decide_check_in(window(), AppointmentState.CONFIRMED, at(9)), CheckInDecision.ACCEPT_CHECK_IN)
    def test_inside_window_accepted(self): self.assertEqual(decide_check_in(window(), AppointmentState.CONFIRMED, at(10,15)), CheckInDecision.ACCEPT_CHECK_IN)
    def test_window_end_routes_to_late_review(self): self.assertEqual(decide_check_in(window(), AppointmentState.CONFIRMED, at(11)),
CheckInDecision.LATE_REVIEW)
    def test_pre_grace_end_routes_to_late_review(self): self.assertEqual(decide_check_in(window(), AppointmentState.CONFIRMED, at(11,14,)),
CheckInDecision.LATE_REVIEW)
    def test_grace_threshold_makes_no_show_eligible_not_automatic(self): self.assertEqual(decide_check_in(window(), AppointmentState.CONFIRMED, at(11,15)),
CheckInDecision.NO_SHOW_ELIGIBLE)
    def test_cancelled_rejected(self): self.assertEqual(decide_check_in(window(), AppointmentState.CANCELLED, at(9,30)), CheckInDecision.REJECT_CANCELLED)
    def test_repeat_checkin_is_idempotent_decision(self): self.assertEqual(decide_check_in(window(), AppointmentState.CHECKED_IN, at(9,30)),
CheckInDecision.ALREADY_CHECKED_IN)
    def test_completed_is_closed(self): self.assertEqual(decide_check_in(window(), AppointmentState.COMPLETED, at(9,30)), CheckInDecision.REJECT_CLOSED)
    def test_naive_current_time_raises(self):
        with self.assertRaises(ValueError): decide_check_in(window(), AppointmentState.CONFIRMED, datetime(2026,10,12,9))
    def test_negative_grace_raises(self):
        with self.assertRaises(ValueError): decide_check_in(window(), AppointmentState.CONFIRMED, at(9), -1)

class TestInvariantI2AppointmentCoherence(unittest.TestCase):
    def test_confirmed_without_checkin_is_valid(self): self.assertEqual(appointment_violations(appointment()), [])
    def test_checkedin_without_timestamp_violates(self): self.assertTrue(appointment_violations(appointment(AppointmentState.CHECKED_IN)))
    def test_checkedin_with_timestamp_is_valid(self): self.assertEqual(appointment_violations(appointment(AppointmentState.CHECKED_IN, at(9,30))), [])
    def test_noshow_with_checkin_violates(self): self.assertTrue(appointment_violations(appointment(AppointmentState.NO_SHOW, at(9,30), at(11,20))))
    def test_noshow_before_grace_violates(self): self.assertTrue(appointment_violations(appointment(AppointmentState.NO_SHOW, None, at(11,14))))
    def test_noshow_after_grace_without_checkin_is_coherent(self): self.assertEqual(appointment_violations(appointment(AppointmentState.NO_SHOW, None,
at(11,15))), [])

class TestInvariantI3Tickets(unittest.TestCase):
    def test_single_active_ticket_has_no_duplicate_violation(self): self.assertEqual(duplicate_active_ticket_appointments([ticket()]), set())
    def test_two_active_tickets_for_one_booking_violate(self):
        ts = [ticket("T-1", number="Q-001"), ticket("T-2", number="Q-002")]
        self.assertEqual(duplicate_active_ticket_appointments(ts), {"A-001"})
        self.assertTrue(ticket_registry_violations(ts))
    def test_withdrawn_ticket_does_not_count_as_active(self):
        ts = [ticket("T-1", state=TicketState.WITHDRAWN, number="Q-001"), ticket("T-2", number="Q-002")]
        self.assertEqual(duplicate_active_ticket_appointments(ts), set())
    def test_ticket_issuance_requires_accepted_checkin(self): self.assertFalse(can_issue_ticket(appointment(), []))
    def test_ticket_issuance_allowed_once_after_checkin(self):
        a = appointment(AppointmentState.CHECKED_IN, at(9,5))
        self.assertTrue(can_issue_ticket(a, []))
        self.assertFalse(can_issue_ticket(a, [ticket()]))
    def test_queue_number_unique_per_session(self):
        ts = [ticket("T-1", "A-001", number="Q-001"), ticket("T-2", "A-002", number="Q-001")]
        self.assertIn("queue_number must be unique within a session", ticket_registry_violations(ts))

class TestInvariantI4NoShowAuthorization(unittest.TestCase):
    def test_authorized_after_threshold_allowed(self): self.assertTrue(can_record_no_show(appointment(), at(11,15), "receptionist"))
    def test_before_threshold_not_allowed(self): self.assertFalse(can_record_no_show(appointment(), at(11,14), "receptionist"))
    def test_unauthorized_role_not_allowed(self): self.assertFalse(can_record_no_show(appointment(), at(11,15), "patient"))
    def test_checkedin_appointment_cannot_be_noshow(self): self.assertFalse(can_record_no_show(appointment(AppointmentState.CHECKED_IN, at(9,5)), at(11,15),
"receptionist"))

class TestInvariantI5TicketTransitions(unittest.TestCase):
    def test_waiting_to_called_allowed(self): self.assertTrue(ticket_transition_allowed(TicketState.WAITING, TicketState.CALLED))
    def test_called_back_to_waiting_allowed_as_recall_correction(self): self.assertTrue(ticket_transition_allowed(TicketState.CALLED, TicketState.WAITING))
    def test_finished_to_waiting_forbidden(self): self.assertFalse(ticket_transition_allowed(TicketState.FINISHED, TicketState.WAITING))
    def test_withdrawn_is_terminal(self): self.assertFalse(ticket_transition_allowed(TicketState.WITHDRAWN, TicketState.CALLED))

if __name__ == "__main__": unittest.main(verbosity=2)
```